

ENGINEERING PLAYBOOK · NO-CODE OVERVIEW

One Command, Always-Fresh SDK

How our auto-generated API client works — and how to drop it into
any React + Spring Boot project that exposes an OpenAPI spec.

Orval

React

Spring Boot

Axios

TanStack Query

Fully automated

A concept-first guide — diagrams and plain English, no code.

Covers the full flow, the one-command automation, common SDK-generation fixes, and the Node-version safeguard.

1 · The idea in one minute

We don't write API code. The backend already describes itself — we turn that description into a ready-to-use, always-correct client.

A **Spring Boot** backend publishes an **OpenAPI spec**: a machine-readable list of every endpoint, its inputs, and its response shapes. Our generator reads that spec and **produces a typed client** for the React app — one ready-made data hook per endpoint. The app simply calls a hook and gets back its data, loading state, and errors. When the backend changes, we re-run **one command** and the whole client folder updates itself.

In one line: the backend's contract is the single source of truth — the frontend client is generated from it, so the two can never silently disagree.

Why teams love it the 30-second case

✓ Zero API code

Every endpoint arrives as a ready hook. Nobody hand-writes requests.

✓ Can't drift

The client is built from the spec, so a backend change surfaces immediately, not in production.

✓ State for free

Loading, errors, caching and refresh are handled automatically.

✓ One command

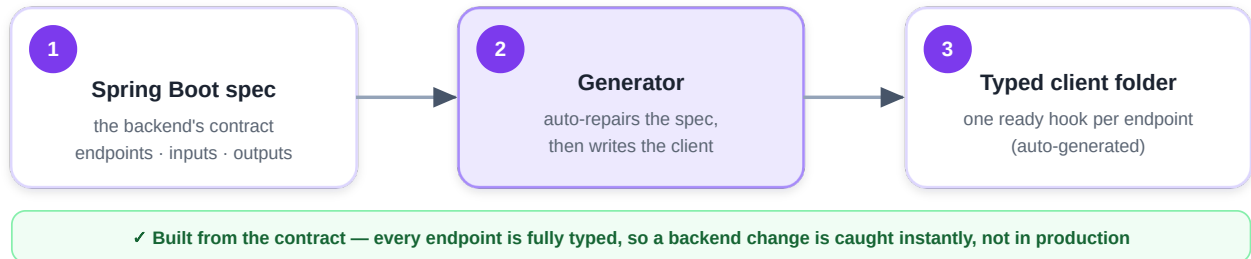
Re-generating the entire client is a single command — the folder updates itself.

2 · How the SDK works — start to end

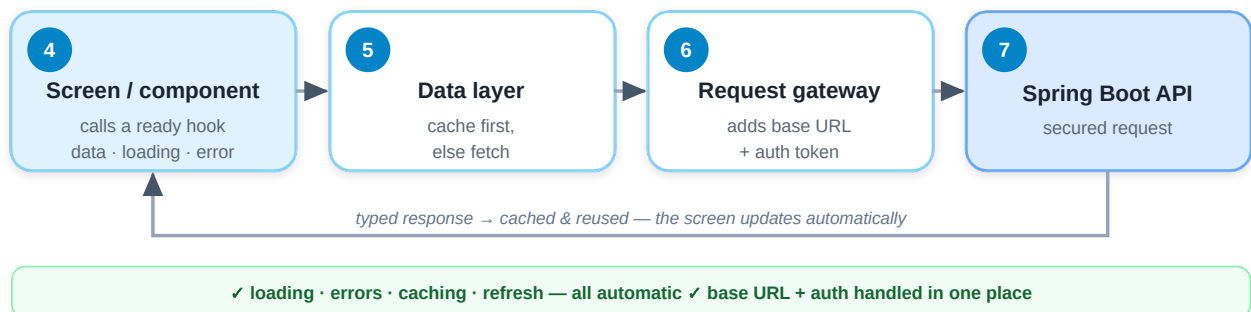
Two timelines: a **generate** step that builds the client, and a **use** path that runs in the browser.

Legend: **generate** happens when you run the command • **use** happens every time a screen opens.

1 · GENERATE — when you run the command

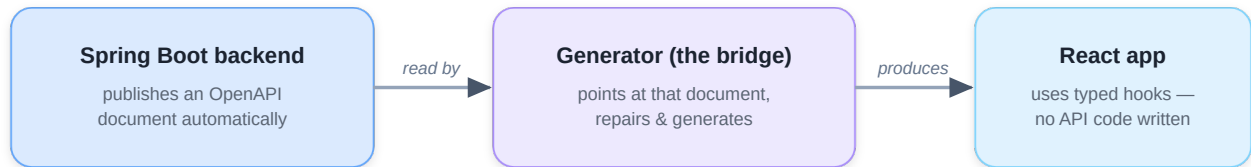


2 · USE — every time a screen opens



3 · Fitting it into any React + Spring Boot project

The same recipe works anywhere the backend can publish an OpenAPI document — which Spring Boot does out of the box.



What to point it at

Project type	Where the contract comes from
Spring Boot (springdoc / Swagger)	The built-in OpenAPI endpoint the backend already exposes.
Any other backend	Any URL or file that serves a valid OpenAPI document.

To adopt it in a new project you only set two things: **where the spec lives** and **where the request gateway lives** (base URL + auth). Everything else — the hooks, the types, the folder structure — is generated for you.

4 · How it fixes the usual SDK-generation pain

Real-world backends rarely emit a perfect spec. Generation normally breaks on small defects. We make it self-healing.

The recurring problems — and how they're absorbed

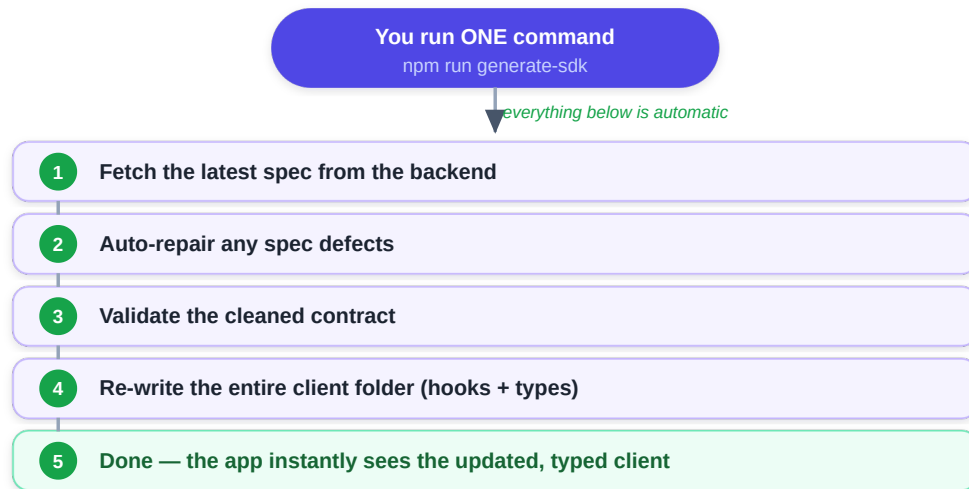
Problem that normally stops generation	How our setup handles it
The spec has small technical defects (common with Spring Boot + Keycloak security setups)	A repair step runs automatically before validation and cleans the known defects — no backend change needed.
A URL expects an ID in its path, but the spec forgot to declare it	The repair step fills in the missing piece so the document validates and generation continues.
The client and the request library "speak different languages" and the types clash	The setup is configured so the generated client and the request gateway match exactly — no type conflicts.
A new backend reveals a brand-new defect	The repair step is extendable : add one small rule for that defect and every future run is covered.

The principle: the repair step only *cleans or fills in* what the contract already implies — it never invents endpoints. So the generated client stays a faithful mirror of the real API.

Honest note: no setup can predict every possible defect up front. Ours is **reactive by design** — when a new backend trips it, you read the clear message it prints and add one small rule. After that, it's automatic forever.

5 · Full automation — one command, the folder updates itself

You run a single command. Behind it, five things happen automatically and the client folder is rewritten end to end.



Why this matters: keeping the client in sync isn't a manual chore. Whenever the backend changes, one command refreshes the whole folder — no copy-pasting, no hand-editing, no missed endpoints.

Optionally, the same single command can be wired to run on a schedule or in the build pipeline, so the client is refreshed without anyone remembering to do it.

6 · The "works on my machine" fix (Node version)

Modern generators need a modern runtime. If a teammate's runtime is too old, generation fails with confusing errors. We made that impossible to get wrong.

What used to go wrong

Someone clones the project on an older runtime, runs the install, and hits a cryptic crash deep inside the tools — with no hint that the real cause is just an out-of-date runtime version.

How we solved it — three automatic safeguards

Safeguard	What it does	When it kicks in
Declare	The project states the minimum runtime version it needs.	Tools & editors read it; warnings appear automatically.
Pin	A small version file fixes the exact runtime line for the team.	One short command switches a teammate to the right version.
Enforce	A friendly check runs during install and stops if the runtime is too old.	Automatically, before anything can break — with clear fix steps.

The result: on the right runtime everything proceeds silently. On the wrong one, the install halts immediately and tells the person exactly how to fix it — instead of failing later with a confusing error.

What a clean first-time setup feels like now

- 1 **Clone the project** and switch to the pinned runtime with one short command.
- 2 **Install** — the safeguard checks the runtime first, then sets everything up.
- 3 **Run the one command** to generate the client folder.
- 4 **Start the app** — the typed hooks are ready to use.

7 · Easy setup, end to end (no code)

The whole thing, as plain steps. Each is a one-time action; after this, day-to-day work is just the single generate command.

- 1 **Make sure the backend publishes its contract.** Spring Boot does this automatically — note the address of its OpenAPI document.
- 2 **Add the runtime safeguards.** Declare the required runtime version, pin it for the team, and add the friendly install-time check. Now nobody can set up the project on the wrong version.
- 3 **Add the request gateway.** One place that knows the backend's base address and attaches the login token to every request.
- 4 **Tell the generator two things:** where the contract lives, and to route every call through the request gateway.
- 5 **Add the auto-repair step** (only if the backend's contract has the usual small defects). It cleans them before generation, automatically.
- 6 **Give the app its data layer.** A single wrapper enables caching, loading and error handling for every hook across the whole app.
- 7 **Run the one command** to generate the client folder — and re-run it any time the backend changes.

From here on: the backend evolves, you run one command, and the entire client updates itself — fully typed, always in sync, with caching and auth handled for you.

The five things to remember

Rule	Why
Never hand-edit the generated folder	It's rewritten on every run — edit the source of truth instead.
The contract is the single source of truth	The client mirrors it, so they can't disagree.
Spec defect? Add one repair rule	Keeps generation automatic forever after.
One command refreshes everything	Staying in sync is never a manual chore.
Pin your runtime version	Removes "works on my machine" failures.